

CHAPTER I

INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

In the modern digital landscape, online advertising has become a vital marketing tool for businesses of all scales. Companies rely on digital ad platforms such as Google Ads, Facebook Ads, and mobile applications to reach targeted audiences and increase their brand visibility. However, the growing dependence on these online platforms has also led to the emergence of serious cybersecurity threats, particularly ad click fraud. Click fraud occurs when an individual, automated script, or malicious bot repeatedly clicks on pay-per-click (PPC) advertisements without any genuine interest in the product or service. This deceptive activity artificially inflates click counts, drains advertisers' budgets, and provides misleading analytics. As a result, it reduces the overall efficiency of digital marketing campaigns and damages the trust between advertisers and ad networks.

The increasing sophistication of fraudulent actors demands a more advanced cybersecurity approach to protect online advertising systems. Traditional fraud detection methods based on simple thresholds or manual analysis have proven insufficient against modern attacks. Cybercriminals now use botnets, VPNs, and click farms to simulate legitimate user behavior, making detection more complex. A robust cybersecurity framework is therefore necessary to ensure data integrity, privacy, and authenticity of online interactions. Integrating security mechanisms such as real-time traffic analysis, anomaly detection, and behavioral monitoring can help in identifying suspicious activities effectively. By combining these cybersecurity techniques with machine learning algorithms, advertisers can proactively defend against click fraud and maintain fair and secure advertising environments.

Artificial Intelligence (AI) and Machine Learning (ML) play a central role in the development of a real-time click fraud detection system. Machine

learning models can analyze large volumes of clickstream data to identify patterns that distinguish between genuine and fraudulent clicks. Algorithms such as decision trees, random forests, and neural networks can learn from historical data and continuously adapt to new fraud behaviors. These models, when integrated into a cybersecurity framework, enable real-time decision-making and automated threat response. AI-based systems can also detect subtle deviations in user behavior, such as unusual click frequency, abnormal IP activity, or repetitive access patterns, which may not be easily identified by traditional systems. This intelligent approach ensures faster detection and minimizes financial losses.

The proposed cybersecurity framework for real-time ad click fraud detection consists of multiple layers designed to ensure security, accuracy, and scalability. The first layer involves data collection and preprocessing, where raw click data from various ad networks are gathered and cleaned. The second layer includes feature extraction and selection, identifying key parameters such as time intervals, user-agent behavior, IP address distribution, and geolocation data. The third layer is the machine learning detection module, which uses trained algorithms to classify clicks as legitimate or fraudulent. The final layer focuses on security enforcement and reporting, where detected frauds trigger alerts, block malicious sources, and update system databases for continuous improvement. This layered approach ensures a comprehensive and adaptive defense mechanism against evolving fraud techniques.

The major advantage of the proposed system lies in its ability to detect fraudulent activities in real time, unlike traditional systems that rely on post-analysis. It enhances the transparency of online advertising by ensuring that every click is authentic and verified. Moreover, the integration of cybersecurity principles ensures data confidentiality and system resilience against manipulation or tampering. The use of AI and ML allows the framework to self-learn and evolve with changing fraud tactics, reducing the dependency on human intervention. Additionally, the system can handle high volumes of ad

traffic efficiently, making it suitable for both large-scale ad networks and small businesses. This proactive detection model not only prevents financial losses but also improves overall campaign performance and advertiser confidence.

In summary, this project aims to design and implement a cybersecurity-based real-time framework capable of identifying and preventing ad click fraud effectively. By leveraging advanced machine learning algorithms and secure data-handling mechanisms, the framework provides an intelligent, automated, and adaptive solution for digital advertising platforms. The project also emphasizes the importance of integrating cybersecurity with data analytics to protect digital ecosystems from evolving cyber threats. In the future, the framework can be extended to detect other types of online advertising fraud such as impression fraud, affiliate fraud, and install fraud. Continuous research and system optimization will further enhance its performance, ensuring secure, transparent, and trustworthy online advertising operations.

1.2 ARTIFICIAL INTELLIGENCE

Artificial Intelligence (AI) is a branch of computer science that focuses on creating machines capable of performing tasks that typically require human intelligence. It involves the development of algorithms and systems that can perceive, reason, learn, and make decisions independently. The concept of AI has evolved from simple rule-based systems to complex neural networks capable of deep learning and cognitive processing. Today, AI is integrated into various fields such as healthcare, finance, education, robotics, and cybersecurity. The goal of AI is not only to simulate human thinking but also to enhance it by providing faster, more accurate, and data-driven solutions. With the explosion of big data and advancements in computing power, AI has transitioned from theoretical research to practical applications that are shaping the digital age.

The development of AI can be traced back to the 1950s when researchers like Alan Turing introduced the idea of machines that could “think.” Over the decades, AI has evolved through multiple stages — from symbolic AI and

expert systems to modern machine learning (ML) and deep learning models. Initially, AI systems relied on predefined logic and rules, but they struggled with complex, unstructured data. The emergence of big data and powerful computational tools enabled a new era of AI that could learn patterns automatically from experience. The 21st century has witnessed massive growth in AI adoption, fueled by cloud computing, open-source frameworks, and data availability. Industries now leverage AI for tasks such as automation, prediction, optimization, and intelligent decision-making, making it one of the most transformative technologies of the modern era.

Artificial Intelligence comprises several core components that work together to simulate intelligent behavior. These include machine learning, natural language processing (NLP), computer vision, expert systems, and robotics. Machine learning allows systems to learn from data without explicit programming, while NLP enables machines to understand and process human language. Computer vision gives machines the ability to interpret visual information, and robotics integrates AI with physical machines for automated control. Additionally, AI systems often employ knowledge representation and reasoning mechanisms to make logical decisions. Together, these components create systems capable of understanding environments, analyzing data, adapting to new conditions, and performing tasks autonomously — from facial recognition to medical diagnosis.

AI has become a cornerstone of innovation across multiple sectors. In healthcare, AI algorithms assist in early disease detection, drug discovery, and patient monitoring. In finance, AI is used for fraud detection, algorithmic trading, and credit risk assessment. Manufacturing industries benefit from AI-driven automation, predictive maintenance, and quality control. In education, AI enables personalized learning and intelligent tutoring systems. Moreover, cybersecurity leverages AI to detect anomalies, prevent intrusions, and enhance data protection in real time. AI also powers everyday technologies such as virtual assistants, recommendation engines, and autonomous vehicles. These

applications highlight how AI not only improves efficiency but also transforms the way industries operate by reducing human error and enabling smarter decision-making.

The adoption of AI offers numerous benefits, including increased accuracy, efficiency, scalability, and cost reduction. It allows organizations to automate repetitive tasks, analyze massive datasets quickly, and make strategic predictions. However, AI also presents significant challenges. Ethical concerns such as bias in algorithms, data privacy, and job displacement have become major topics of discussion. Additionally, AI systems require high-quality data and computational resources, making implementation costly for small organizations. There are also risks related to over-dependence on automation and lack of transparency in decision-making processes, known as the “black box problem.” Balancing innovation with ethics and governance is crucial to ensure that AI serves humanity responsibly and effectively.

The future of Artificial Intelligence promises even greater advancements and deeper integration into everyday life. Emerging technologies such as explainable AI (XAI), quantum AI, and autonomous intelligent systems are set to redefine how machines interact with humans and the environment. AI is expected to drive the next wave of industrial transformation, often referred to as Industry 5.0, where human creativity and machine intelligence collaborate harmoniously. Governments and organizations worldwide are investing in AI research to develop safer, fairer, and more transparent systems. As AI continues to evolve, its applications will extend beyond automation to include emotional intelligence, self-learning systems, and ethical reasoning. The ongoing challenge will be ensuring that AI remains secure, trustworthy, and beneficial for the collective progress of society.

1.3 BACKGROUND OF THE STUDY

In recent years, digital advertising has become one of the most powerful and widely used marketing tools across the globe. Businesses of all sizes invest heavily in online advertising campaigns to attract customers, promote products,

and increase brand visibility. Platforms such as Google Ads, Facebook, and YouTube have revolutionized the advertising industry through their pay-per-click (PPC) models, where advertisers pay only when a user clicks on an ad. However, this digital ecosystem, while highly efficient, has also become a lucrative target for cybercriminals. The ease of automation, accessibility of online systems, and high financial turnover have collectively created opportunities for click fraud, a major threat to the integrity and effectiveness of online advertising.

Click fraud refers to the malicious or deceptive clicking of online advertisements with no real intention of engagement or purchase. This activity can be performed by individuals, automated bots, or organized groups known as click farms. The objective of click fraud is often to inflate advertising costs for competitors or to generate illegitimate revenue for website owners hosting the ads. Over time, fraudulent techniques have evolved in complexity — attackers now use advanced bots, virtual private networks (VPNs), and fake user agents to mimic real human behavior. As a result, distinguishing between genuine and fraudulent clicks has become a challenging task for advertisers and ad networks. This growing sophistication has made click fraud not just a marketing problem but a serious cybersecurity concern that demands intelligent and automated detection mechanisms.

Traditional click fraud detection methods primarily rely on rule-based systems, IP filtering, or threshold-based monitoring to identify abnormal click patterns. While these techniques can detect basic fraudulent behaviors, they fail to adapt to rapidly changing attack strategies. Cybercriminals continuously modify their tactics by randomizing clicks, changing IP addresses, and using distributed bots to appear legitimate. Moreover, most existing systems analyze click data after the fraud has already occurred, which means advertisers suffer losses before detection. This reactive approach leads to inefficiency, data inaccuracy, and wasted ad expenditure. Hence, there is a pressing need for a

real-time, intelligent, and secure framework that can proactively detect and prevent fraudulent clicks before they cause financial or reputational damage.

To effectively address this issue, integrating cybersecurity principles with machine learning (ML) techniques has emerged as a promising solution. Cybersecurity ensures that data integrity, authenticity, and system protection are maintained, while machine learning provides adaptive and automated capabilities for recognizing complex fraud patterns. ML models can analyze historical click data, detect anomalies in user behavior, and classify traffic as legitimate or fraudulent based on learned patterns. By combining these approaches, a cybersecurity framework for real-time ad click fraud detection can provide continuous monitoring, intelligent threat identification, and rapid response mechanisms. Such a framework not only enhances accuracy but also strengthens the security posture of digital advertising networks.

Real-time detection plays a crucial role in preventing the financial impact and reputational harm caused by click fraud. Instead of relying on post-campaign analysis, a real-time system can identify fraudulent activity as it happens, allowing advertisers and ad platforms to take immediate corrective action. This proactive approach helps in minimizing losses, maintaining data accuracy, and ensuring fair competition in online advertising. Real-time detection frameworks utilize high-speed data processing, behavioral analytics, and automated alerts to detect and mitigate fraudulent actions instantly. Furthermore, such systems contribute to the overall cyber resilience of the digital marketing ecosystem by continuously learning from new fraud patterns and adapting to emerging threats.

In summary, the increasing dependence on online advertising has led to a parallel rise in click fraud, posing serious challenges to advertisers and cybersecurity professionals. Existing detection methods lack the intelligence and speed required to combat modern, sophisticated attacks. The integration of AI-driven machine learning models within a cybersecurity-based real-time framework offers a viable and innovative solution to this problem. This study

aims to develop such a framework capable of identifying, analyzing, and preventing ad click fraud effectively and efficiently. By addressing both the technical and security aspects of click fraud detection, this research contributes to creating a safer, more transparent, and trustworthy digital advertising environment.

1.4 OBJECTIVES

- To design and develop a cybersecurity-based framework capable of detecting ad click fraud in real time.
- To analyze user behavior patterns and identify abnormal or suspicious click activities using machine learning techniques.
- To integrate cybersecurity principles such as data integrity, authentication, and encryption to ensure secure detection and reporting.
- To implement machine learning algorithms that can classify clicks as legitimate or fraudulent with high accuracy.
- To reduce financial losses and improve the return on investment (ROI) for advertisers through early fraud detection.
- To build a scalable and efficient detection system capable of handling large volumes of ad traffic.
- To evaluate system performance in terms of accuracy, precision, recall, and real-time responsiveness.

1.5 PROBLEM STATEMENT

The rapid expansion of digital advertising has made online platforms a major source of revenue for businesses. However, this growth has also given rise to ad click fraud, a serious cybersecurity threat that undermines the credibility and financial efficiency of online marketing campaigns. Click fraud occurs when bots, scripts, or malicious users generate false clicks on pay-per-click (PPC) advertisements without genuine user interest. This artificial activity inflates the number of clicks, leading advertisers to pay for non-legitimate engagement. As a result, companies suffer financial losses, distorted performance metrics, and reduced trust in digital advertising networks.

Traditional fraud detection methods, such as rule-based systems and manual analysis, are no longer sufficient to handle the complexity of modern click fraud techniques. Cybercriminals now use sophisticated tools like botnets, VPN masking, and randomized click patterns to mimic legitimate user behavior. These tactics make it extremely difficult for conventional systems to differentiate between genuine and fraudulent clicks. Moreover, most existing detection mechanisms operate after the fraudulent activity has already occurred, making them reactive rather than preventive. This delayed detection leads to wasted advertising budgets and poor campaign performance.

The absence of a real-time and intelligent cybersecurity framework capable of identifying fraudulent clicks instantly remains a major challenge in the online advertising industry. Current systems lack integration between cybersecurity measures and machine learning-based predictive analytics, which limits their effectiveness in responding to evolving threats. There is an urgent need for a comprehensive system that can analyze user behavior, detect anomalies, and respond proactively to prevent losses. Such a framework should ensure data integrity, security, and accuracy while maintaining high performance and scalability.

Therefore, the core problem addressed by this study is the lack of an efficient and secure mechanism for real-time detection and prevention of ad click fraud. The proposed research aims to develop a cybersecurity framework integrated with machine learning algorithms to identify and mitigate fraudulent clicks dynamically. By bridging the gap between data analysis and cybersecurity, this project seeks to enhance the reliability, transparency, and safety of digital advertising ecosystems.

1.6 SCOPE

The scope of this study focuses on the design and implementation of a cybersecurity-based real-time framework for detecting and preventing ad click fraud in digital advertising platforms. The research aims to integrate machine learning algorithms and cybersecurity techniques to accurately identify

fraudulent clicks while maintaining data integrity and system reliability. The study covers various aspects of ad traffic analysis, including user behavior monitoring, anomaly detection, and classification of legitimate versus fraudulent clicks. It emphasizes the development of a system capable of processing large-scale ad data efficiently and generating instant alerts when suspicious activities are detected. The framework is designed to operate in a real-time environment, ensuring continuous monitoring and immediate response to threats. Furthermore, it focuses on improving advertisers' return on investment (ROI) and maintaining transparency in online ad networks.

CHAPTER II

LITERATURE REVIEW

2.1 Alzahrani, R.A., Aljabri, M. and Mohammad, R.M.A., 2025. Ad click fraud detection using machine learning and deep learning algorithms. *IEEE Access*.

In online advertising, click fraud poses a significant challenge, draining budgets and threatening the industry's integrity by redirecting funds away from legitimate advertisers. Despite ongoing efforts to combat these fraudulent practices, recent data emphasizes their widespread and persistent nature. Toward detecting click fraud effectively, this study employed a comprehensive feature engineering and extraction approach to identify subtle differences in click behavior that could be used to distinguish fraudulent from legitimate clicks. Subsequently, a thorough evaluation was conducted involving nine diverse machine learning (ML) and Deep Learning (DL) models. After (RFE), the ML models consistently demonstrated robust performance. DT and RF surpassed 98.99% accuracy, while GB, LightGBM, and XGBoost achieved 98.90% or higher. Precision scores, measuring accurate identification of fraudulent clicks, exceeded 98% for models like ANN. In parallel, deep learning (DL) models, including Convolutional Neural Network (CNN), Deep Neural Network (DNN), and Recurrent Neural Network (RNN), showcased strong performance. RNN, in particular, achieved 97.34% accuracy, emphasizing its efficacy. The study underscores the prowess of tree-based methods and advanced algorithms in detecting click fraud, as evidenced by high accuracy, precision, and recall scores. These findings contribute valuable insights to combat click fraud and establish the groundwork for the strategic development of anti-fraud measures in online advertising.

2.2 Aljabri, M. and Mohammad, R.M.A., 2023. Click fraud detection for online advertising using machine learning. *Egyptian Informatics Journal*, 24(2), pp.341-350.

Advertising corporations have moved their focus to online and in-App advertisements in response to the expansion of digital technologies and social media. Online advertising represents the primary revenue source for advertising networks that serve as the middlemen between advertisers and advertisement publishers. The advertising networks pay the publisher of the advertisement based on the number of clicks through to advertisers, following the pay-per-click (PPC) payment scheme. However, there is a growing security issue with this payment approach known as Click Fraud. Click fraud is the illegal process of clicking on pay-per-click advertisements to increase publishers' revenue or deplete advertisers' budgets. Artificial intelligence techniques have been increasingly employed to solve complicated challenges in different research areas, including cybersecurity, to achieve unexpected outcomes. In this paper, several Machine Learning models were constructed to establish whether the user is a human or a bot and conducted a comparative performance analysis using a set of evaluation metrics. We used a real captured dataset detailing Internet users' behavior while navigating websites. We extracted a set of features related to users' behaviors, including the number of webpages viewed during the browsing session, the duration of the browsing journey, and the actions performed. The empirical results revealed that all the considered models obtained good results, where the random forest algorithm surpassed all other algorithms in all evaluation metrics.

2.3 Sisodia, Deepti, and Dilip Singh Sisodia. "A transfer learning framework towards identifying behavioral changes of fraudulent publishers in pay-per-click model of online advertising for click fraud detection." *Expert Systems with Applications* 232 (2023): 120922.

The absence of a publicly available user-click dataset makes the task of fraudster identification particularly challenging to detect click fraud in online advertising. However, the task becomes more complicated with concept drift, where a publisher appears in distinct sets with the same pattern but differs in real status labels. Fraud-detection algorithms developed the actual status labels.

The reliability of the predictions made by learning models needs to be examined to deal with the concept drift concerning the changing behaviour of fraudulent publishers without re-training the predictive model from scratch. However, the scarcity of pre-trained models aggravates the issue. Thus, we proposed a 1D-convolutional neural network-based FINet that deals with such challenges using a model trained on a correlated prediction modelling problem using transfer learning. To overcome the scarcity of pre-trained models, we designed a model FINet trained on the correlated dataset Talking Data Ad TDA and utilized the weights of the trained model to learn a model on the FDMA2012 dataset better. This predicted modelling on a distinct but related problem is re-used for accelerating the training and improving the FINet's fraudster identification performance. The proposed FINet's behaviour is investigated based on eight optimizer algorithms in terms of Average Precision, Recall, F1-score, and AUC scores and performance generalization is verified by conducting experiments with existing state-of-art deep learning models. The implementation results are notably higher than the existing state-of-the-art deep learning models and exhibit effective performance towards fraudster's identification in detecting click fraud.

2.4 Subburayan, Baranidharan, David Winster, K. Dhanalakshmi, and R. Rajkumar. "Combating Evolving Threats: A Systematic Review of Online Ad Fraud Detection." *Available at SSRN 5263103* (2025).

This systematic review investigates online ad fraud detection and brand safety research from 2011 to 2024. The analysis reveals a continuous battle against click fraud and its evolving forms. Machine learning has become a cornerstone of detection efforts, offering superior capabilities compared to traditional methods. The rise of mobile advertising necessitated the development of specialized solutions to address distinct user behavior and data patterns on this platform. However, research highlights an expanding threat landscape beyond click fraud, encompassing impression fraud and placement fraud. Brand safety concerns have also gained prominence, emphasizing the

importance of protecting brand reputation. The review underscores the need for collaboration between researchers and industry professionals to achieve a more secure and trustworthy online advertising ecosystem.

2.5 Batool, A., & Byun, Y. C. (2022). An ensemble architecture based on deep learning model for click fraud detection in pay-per-click advertisement campaign. *IEEE Access*, 10, 113410-113426.

With the rapid development of online advertising, click fraud is a serious issue for the internet market. Click fraud is a dishonest attempt to improve a website's profit or deplete an advertiser's budget by clicking on pay-per-click advertisements. For an extended period, this illegal act has a threat to the industrial sectors. As a result, these businesses hesitate to advertise their items on mobile apps and websites, as numerous groups attempt to take advantage of them. To safely advertise their services and products online, a robust mechanism is needed for efficient click fraud detection. To tackle this issue, an ensemble architecture of machine learning and deep learning is proposed to detect click fraud in online advertisement campaigns. The proposed ensemble architecture consists of a Convolutional Neural Network (CNN), and a BiLSTM is used to extract hidden features, while the Random Forest (RF) is used for classification. The main objective of the proposed research study is to develop a hybrid DL model for automatic feature extraction from clicks data and then process through an RF classifier into two classes, such as fraudulent and non-fraudulent clicks. Furthermore, a preprocessing module is developed to preprocess data by dealing with categorical attributes and imbalanced data to enhance the reliability and consistency of the clicks data. In addition, different evaluation criteria are used to evaluate and compare the performance of the proposed CNN-BiLSTM-RF with the ensemble and standalone models.

2.6 Alzahrani, Reem A., and Malak Aljabri. "AI-based techniques for Ad click fraud detection and prevention: Review and research directions." *Journal of Sensor and Actuator Networks* 12, no. 1 (2022): 4.

Online advertising is a marketing approach that uses numerous online channels to target potential customers for businesses, brands, and organizations. One of the most serious threats in today's marketing industry is the widespread attack known as click fraud. Traffic statistics for online advertisements are artificially inflated in click fraud. Typical pay-per-click advertisements charge a fee for each click, assuming that a potential customer was drawn to the ad. Click fraud attackers create the illusion that a significant number of possible customers have clicked on an advertiser's link by an automated script, a computer program, or a human. Nevertheless, advertisers are unlikely to profit from these clicks. Fraudulent clicks may be involved to boost the revenues of an ad hosting site or to spoil an advertiser's budget. Several notable attempts to detect and prevent this form of fraud have been undertaken. This study examined all methods developed and published in the previous 10 years that primarily used artificial intelligence (AI), including machine learning (ML) and deep learning (DL), for the detection and prevention of click fraud. Features that served as input to train models for classifying ad clicks as benign or fraudulent, as well as those that were deemed obvious and with critical evidence of click fraud, were identified, and investigated. Corresponding insights and recommendations regarding click fraud detection using AI approaches were provided.

2.7 Lyu, Q., Li, H., Zhou, R., Zhang, J., Zhao, N. and Liu, Y., 2022. BCFDPS: A Blockchain-Based Click Fraud Detection and Prevention Scheme for Online Advertising. *Security and Communication Networks*, 2022(1), p.3043489.

Online advertising, which depends on consumers' click, creates revenue for media sites, publishers, and advertisers. However, click fraud by criminals, i.e., the ad is clicked either by malicious machines or hiring people, threatens this advertising system. To solve the problem, many schemes are proposed which are mainly based on machine learning or statistical analysis. Although these schemes mitigate the problem of click fraud, several problems still exist. For

example, some fraudulent clicks are still in the wild since their schemes only discover the fraudulent clicks with a probability approaching but not 100%. Also, the process of detecting a click fraud is executed by a single publisher, which makes a chance for the publisher to obtain illegal income by deceiving advertisers and media sites. Besides, the identity privacy of consumers is also exposed because the schemes deal with the plain text of consumers' real identity. Therefore, in this paper, a blockchain-based click fraud detection and prevention scheme (BCFDPS) for online advertising is proposed to deal with the above problems. Specifically, the BCFDPS mainly introduces bilinear pairing to implicitly verify whether a consumer's real digital identity is contained in a click message to significantly avoid click fraud and employs a consortium blockchain to ensure the transparency of the detection and prevention process. In our scheme, the clicks by machines or fraud ones by a human can be accurately detected and prevented by media sites, publishers, and advertisers. Furthermore, ciphertext-policy attribute-based encryption is adopted to protect the identity privacy of consumers.

2.8 Sadeghpour, Shadi, and Natalija Vljajic. "Click fraud in digital advertising: A comprehensive survey." *Computers* 10, no. 12 (2021): 164.

Recent research has revealed an alarming prevalence of click fraud in online advertising systems. In this article, we present a comprehensive study on the usage and impact of bots in performing click fraud in the realm of digital advertising. Specifically, we first provide an in-depth investigation of different known categories of Web-bots along with their malicious activities and associated threats. We then ask a series of questions to distinguish between the important behavioral characteristics of bots versus humans in conducting click fraud within modern-day ad platforms. Subsequently, we provide an overview of the current detection and threat mitigation strategies pertaining to click fraud as discussed in the literature, and we categorize the surveyed techniques based on which specific actors within a digital advertising system are most likely to deploy them. We also offer insights into some of the best-known real-world

click bots and their respective ad fraud campaigns observed to date. According to our knowledge, this paper is the most comprehensive research study of its kind, as it examines the problem of click fraud both from a theoretical as well as practical perspective.

2.9 Keserwani, Pankaj Kumar, Mahesh Chandra Govil, and Emmanuel Shubhakar Pilli. "The web ad-click fraud detection approach for supporting to the online advertising system." *International Journal of Swarm Intelligence* 7, no. 1 (2022): 3-24.

With the introduction of e-commercial activities, revenue is generated in several legal and illegal ways. Marketing of a product or service in an online environment is increasing day by day since most users are getting involved in an online environment for buying or selling items. As a result, attackers are getting more space for performing online attacks by different malicious activities. The advertisement (ad) fraud, which is increasing exponentially with each passing day, is one of them. Ad fraud causes defame for the online advertising system and low return on the advertiser's investment (ROI). The paper reveals a methodology for detecting web ad-click frauds so that the ROI of an advertiser can be increased. The developed algorithm to detect the ad-click frauds in the proposed methodology can detect the web ad-click frauds. The results of the ad-click fraud algorithm are verified with the help of popular machine learning (ML) algorithms - k-nearest neighbour (k-NN), random forest (RF), decision tree (DT), logistic regression (LR) and support vector machine (SVM), with accuracies achieved.

2.10 Abbas, Z.A., Hilal, Z.M. and Jabbar, H.G., 2025. Click Fraud Detection in Online Advertising: A Comparative Study of Machine Learning Models. *International Journal of Safety & Security Engineering*, 15(3).

Smartphone advertisement is increasingly used among many applications and allows developers to obtain revenue through in-app advertising. Our study aims at identifying potential security risks of mobile-based advertising services

where advertisers are charged for their advertisements on mobile applications. In the Android platform, we particularly implement bot programs that can massively generate click events on advertisements on mobile applications and test their feasibility with eight popular advertising networks. Our experimental results show that six advertising networks (75 %) out of eight are vulnerable to our attacks. To mitigate click fraud attacks, we suggest three possible defense mechanisms: (1) filtering out program-generated touch events; (2) identifying click fraud attacks with faked advertisement banners; and (3) detecting anomalous behaviors generated by click fraud attacks. We also discuss why few companies were only willing to deploy such defense mechanisms by examining economic misincentives on the mobile advertising industry.

CHAPTER III

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the existing digital advertising ecosystem, ad clicks are often considered genuine and are directly used to measure campaign performance. Most advertising platforms rely on the assumption that every click represents authentic user engagement, which forms the basis for calculating metrics such as click-through rate (CTR) and return on investment (ROI). However, this assumption overlooks the growing complexity of fraudulent activities in online advertising.

Traditional click monitoring systems typically use manual observation or simple threshold-based methods, such as blocking IP addresses that generate excessive clicks within a short time frame. While these methods can identify some obvious anomalies, they are not equipped to detect more sophisticated forms of click fraud. As fraudsters evolve, they employ advanced techniques that can easily bypass these basic filters, making traditional systems ineffective in maintaining data integrity.

3.1.1 DISADVANTAGES

- High false positives/negatives.
- Cannot detect intelligent or disguised fraud.
- No real-time detection or data-driven prediction.

3.2 PROPOSED SYSTEM

The proposed system introduces an AI-powered Click Fraud Detection Model that leverages advanced Machine Learning (ML) techniques to accurately identify fraudulent click activities in digital advertising. Unlike traditional rule-based approaches, this intelligent system focuses on learning complex patterns and behaviors from data to distinguish between genuine and fraudulent user interactions. By applying data-driven intelligence, the model

enhances reliability, reduces manual intervention, and ensures fair ad performance measurement for advertisers.

The system performs a comprehensive analysis of multiple parameters, including user behavior, click frequency, IP address activity, device information, and time-based patterns. These features collectively help in understanding the context and consistency of user clicks. For example, unusual spikes in clicks from a single region, repetitive device signatures, or abnormal session durations can signal fraudulent activity. This multi-factor analysis allows the model to capture even the most subtle irregularities that static monitoring systems fail to detect.

A trained ML model, such as Random Forest, serves as the core component of this system. The algorithm is trained using large datasets containing both legitimate and fraudulent click records. Through this learning process, the model identifies complex, non-linear relationships and feature interactions that help differentiate between normal and suspicious click behavior. This results in high predictive accuracy, significantly improving the reliability of click validation processes in advertising campaigns.

Performance evaluation of the proposed model demonstrates impressive results, achieving over 98% accuracy in identifying fraudulent clicks. Such precision ensures that advertisers can trust the system's predictions to make data-driven marketing decisions and optimize campaign budgets effectively. The high accuracy rate not only reduces financial losses but also improves advertiser confidence by offering transparent and verifiable fraud detection insights.

To make the system user-friendly, a web-based interface is developed using the Django framework. Through this interface, users can easily upload datasets or provide ad links for real-time fraud analysis. The platform instantly displays prediction results, along with a risk score, an explanation of classification results, and the top influencing features that led to the decision. This interactive and explainable AI approach ensures that users not only get accurate results but

also understand the reasoning behind each prediction, making the system both effective and transparent.

3.2.1 ADVANTAGES

- Automated and data-driven fraud detection.
- Reduces financial losses for advertisers.
- Easy to integrate into ad platforms.
- Provides real-time insights through an interactive dashboard.

3.3 MODULES

3.3.1 DATASET

The Dataset Module is a crucial component of the system responsible for handling all user-provided input data in a structured and efficient manner. It primarily deals with CSV files, which can be uploaded directly by the user or accessed through a provided link. Once the CSV is received, the module reads the data and organizes it into a standardized format, ensuring that all necessary fields are correctly identified and processed. This preprocessing step is essential to maintain data integrity and consistency before feeding the data into subsequent modules for analysis or prediction.

The module focuses on capturing key features relevant to the application, including `click_time`, which records the exact timestamp of user interactions, URL link, which identifies the target web address, referral source, indicating where the user arrived from, and IP address, which helps in tracing and analyzing traffic patterns. Each of these features is extracted and validated to ensure that missing or malformed data does not affect model performance.

Beyond simple extraction, the Dataset Module also performs essential preprocessing tasks such as formatting timestamps, normalizing URLs, encoding categorical fields, and handling missing values. This preparation ensures that the data conforms to the model's expected input structure, enabling accurate and reliable predictions. By centralizing all data handling operations,

the module simplifies downstream processing and improves the efficiency of the overall system.

3.3.2 ML

The ML Model Module is the core component of the system responsible for analyzing click data and making predictions regarding potential fraudulent activity. This module is designed to train machine learning models, such as Random Forest or XGBoost, using the structured click dataset prepared by the Dataset Module. During the training process, the module learns patterns and relationships between features such as `click_time`, URL, referral, and IP address, enabling it to distinguish between legitimate and suspicious clicks effectively.

Once the model is trained, the module saves the trained model to disk using formats like joblib or .pkl, ensuring that it can be reused later for real-time prediction without retraining. This persistence allows for faster deployment and reduces computational overhead, especially when handling large volumes of click data. The module is also capable of making predictions on new incoming click records, classifying each click as either Fraudulent or Legitimate, based on the patterns learned during training.

In addition to binary classification, the ML Model Module calculates a probability or confidence score, which indicates the level of risk associated with each click. This score provides valuable insight into the likelihood of fraudulent behavior, allowing downstream systems or analysts to prioritize high-risk clicks for further investigation. By integrating training, prediction, and risk scoring into a single cohesive module, the ML Model Module ensures that the system can operate efficiently, accurately, and reliably in real-time environments.

3.3.3 VIEWS

The Views Module serves as the interface between the user and the backend system, managing how data is received, processed, and presented. Its primary view functions, such as `upload_click(request)` or `paste_link(request)`, allow users to either upload a CSV file containing click data or submit an individual ad link for analysis. Upon receiving the input, the module immediately invokes the Dataset Module to extract relevant features like `click_time`, URL, referral source, and IP address, ensuring that the data is properly structured and ready for prediction.

After feature extraction, the Views Module calls the ML Model Module to classify each click as either Fraudulent or Legitimate. Alongside the binary prediction, it also retrieves the probability or confidence score, indicating the likelihood of fraud. To make the results more interpretable, the module can provide reasons or explanations for the prediction, which may be rule-based (e.g., known blacklisted IPs) or feature-based (e.g., abnormal click patterns). Additionally, it identifies the top features contributing to the prediction, helping users and analysts understand which factors most influenced the model's decision.

Finally, the Views Module prepares a context dictionary containing all these outputs and passes it to the web template, allowing for a clear and user-friendly presentation on the frontend. By orchestrating data extraction, prediction, and explanation within a single module, the Views Module ensures a seamless and efficient workflow from user input to actionable insights, making real-time fraud detection accessible and understandable.

3.3.4 TEMPLATE

The Templates Module is responsible for the frontend presentation of the system, providing a user-friendly interface through HTML templates such as `upload_click.html` or `paste_link.html`. These templates contain forms that allow users to either upload a CSV file of click data or input an

individual ad link directly. The templates are designed to be intuitive, guiding users through the submission process while ensuring that the input data is correctly captured for processing by the backend modules.

Once the backend processes the input, the templates dynamically display the results to the user. This includes the click status, indicating whether the submitted data is classified as Fraudulent or Legitimate, along with the associated probability or confidence percentage, which helps users understand the risk level. The templates can also show reasons for the prediction, either through rule-based or feature-based explanations, providing transparency and interpretability of the model's decisions. Additionally, the templates highlight the top features or metrics that contributed to the prediction, giving users actionable insights into the factors driving the model's classification.

By combining input functionality with clear visualization of results, the Templates Module ensures that the system is both accessible and informative. It bridges the gap between complex backend computations and user comprehension, enabling users to quickly interpret predictions and make data-driven decisions regarding ad click activity.

3.3.5 URL

The URLs Module is responsible for routing incoming HTTP requests to the appropriate views in the system. It defines clear and structured URL patterns that allow users to access the system's functionalities seamlessly. For example, the route `/upload/` is linked to the view that handles CSV file uploads and triggers the prediction pipeline, while `/paste/` is mapped to the view that accepts a single ad link for analysis. By centralizing all URL routing in this module, the system ensures maintainability, readability, and easy navigation, allowing developers to expand or modify routes without disrupting the overall workflow.

The Static or Frontend Module enhances the user experience by providing styling, interactivity, and visual feedback. CSS files in this module ensure a clean, professional, and responsive user interface, making forms, tables, and results visually appealing. JavaScript can be used to create dynamic elements such as progress bars showing fraud probability, interactive charts to visualize click patterns, or badges indicating the status of a click as Good or Fraudulent. This module bridges the gap between backend predictions and user comprehension, delivering an intuitive and engaging interface for users to interact with the system.

CHAPTER IV

SYSTEM DESIGN

4.1 ARCHITECTURE DIAGRAM

An architecture diagram provides a high-level visual representation of a system's structure, showing how its components interact with each other. It typically illustrates the major modules, layers, and interfaces, including servers, databases, APIs, and user interfaces, highlighting their relationships and communication pathways. This diagram helps stakeholders understand the system's design, deployment strategy, and overall organization, making it easier to plan development, identify potential bottlenecks, and ensure scalability and maintainability. The workflow begins with the collection of datasets in the form of CSV files. These datasets contain important information such as user IDs, IP addresses, URLs, referrers, time stamps, and user-agent details. This data is gathered from click logs or ad servers and forms the foundation of the entire system. The quality of this dataset is very important because the model's accuracy depends on how well the data represents real-world user behavior.

Once the dataset is collected, the next stage is Pre-Processing. In this phase, the raw data is cleaned and prepared for machine learning. Steps include removing duplicate and irrelevant records, filling or removing missing values, and formatting the data into a uniform structure. Data encoding is also carried out to convert categorical variables such as browser type or country into numerical form. Normalization and scaling are performed to bring all data values into a similar range. These steps help improve model stability and performance during training.

The next phase is Feature Engineering, where useful features are created from the available data. This step helps the model understand complex relationships in the data. For example, features such as the number of clicks per user, time

difference between consecutive clicks, or the frequency of clicks from the same IP address can provide strong indicators of fraudulent activity. The aim of feature engineering is to identify and use only those variables that have a direct impact on the prediction outcome. Proper feature selection also reduces training time and improves overall model accuracy.

After the features are finalized, the Random Forest model is trained using the processed dataset. Random Forest is an ensemble learning technique that builds multiple decision trees and combines their outputs to make a final prediction. It is known for its robustness and ability to handle both numerical and categorical data effectively. During this stage, the model learns the difference between legitimate and fraudulent clicks based on past examples. The trained model is then tested using a validation dataset to check its performance in terms of accuracy, precision, and recall.

Once the model performs satisfactorily, the Model Saving step is carried out. The trained Random Forest model is saved using file formats such as .pkl or .joblib, so it can be reused later without retraining. The saved model is then Integrated with the User Interface (UI) to make the system interactive and easy to use. Through this interface, a user can upload new click data, view predictions, and analyze results.

Finally, during the Prediction stage, the uploaded data is processed by the saved model to identify fraudulent activities. The output includes the risk level, probability of fraud, and the main features that influenced the prediction. This provides clear and interpretable results for decision-making. Overall, this step-by-step workflow ensures efficient detection of click fraud using the Random Forest algorithm and helps maintain the integrity of digital advertising systems

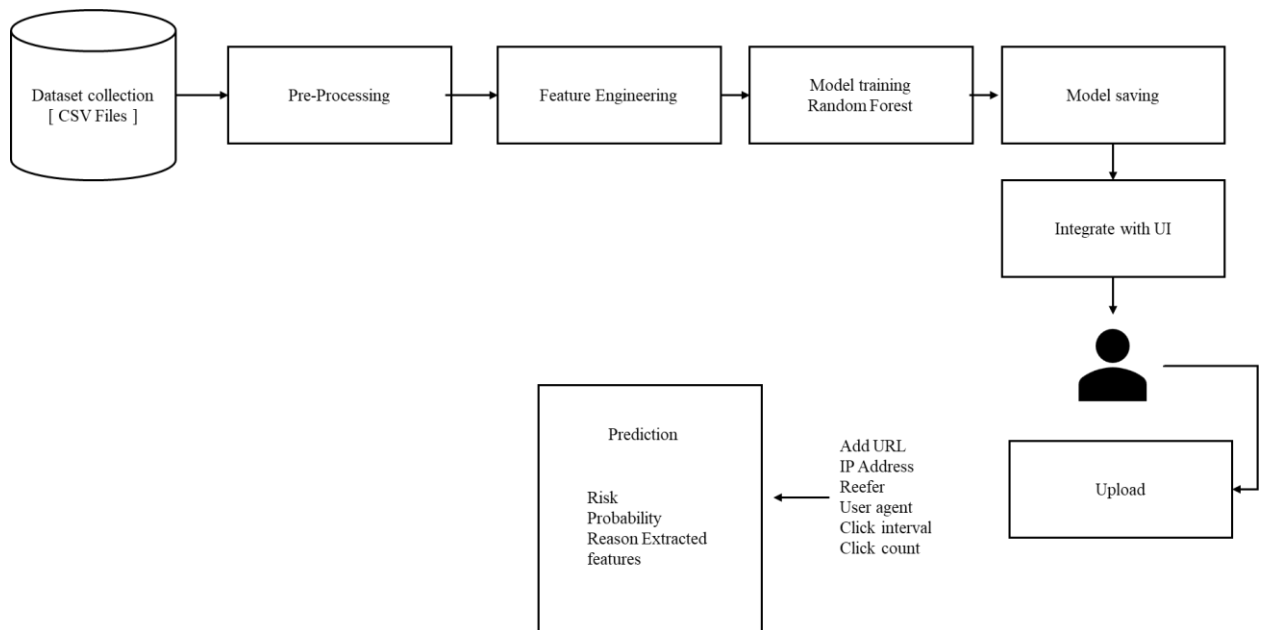


Fig 4.1 Architecture Diagram

4.2 DATA FLOW DIAGRAM

A data flow diagram depicts the flow of information within a system, showing how data moves between processes, data stores, and external entities. It uses standardized symbols to represent processes, inputs, outputs, and storage, providing a clear view of how data is processed and transformed. DFDs are valuable for analyzing system requirements, identifying inefficiencies, and ensuring that all data interactions are properly captured, enabling developers and analysts to design robust and efficient systems.

Level 1

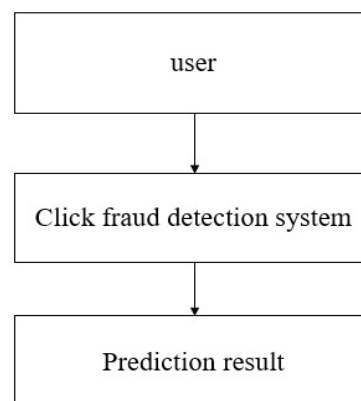


Fig 4.2 Data flow Diagram Level 1

This diagram represents a simple Level 1 overview of a click fraud detection system. At the top, the user interacts with the system, typically by clicking on an online advertisement. The user's clicks are then analyzed by the click fraud detection system, which monitors patterns such as unusual click frequency, IP address inconsistencies, or suspicious user behavior. The system processes this data using algorithms or machine learning models to determine whether the clicks are genuine or fraudulent. Finally, the system produces a prediction result, indicating whether the detected clicks are legitimate or likely part of a click fraud attempt. This level of diagram shows the basic flow of information without diving into the internal components of the detection system.

Level 2

This flowchart illustrates the architecture of a machine learning system, starting from user input through a web interface, followed by feature extraction and pre-processing/encoding of data before it is fed into a machine learning model, specifically a Random Forest Classifier (RFC). The model's output is evaluated by a rule-based analyzer, and the final results are displayed to the user, establishing a clear workflow from initial data collection to end-user feedback.

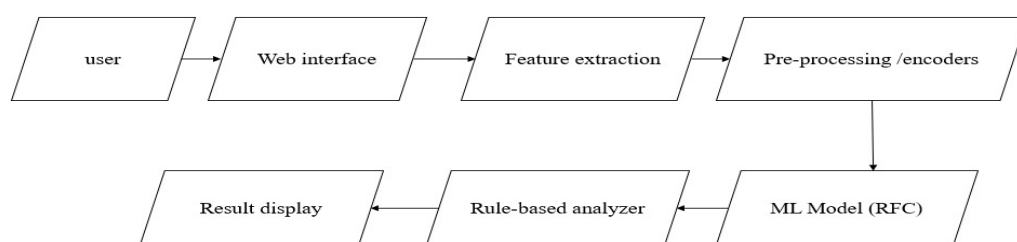


Fig 4.3 Data flow Diagram Level 2

Level 3

This diagram presents a system where user input data undergoes feature extraction—gathering metrics like click count, interval, domain checks, user

agent, referrer, bytes sent/received, and duration—before being encoded and preprocessed for a Random Forest machine learning model. The model's prediction, probability, and reasoning, along with the analyzed features, are sent to a rule-based analyzer that flags patterns such as high frequency, bot-like user agents, or suspicious domains, with results then displayed to users for final interpretation.

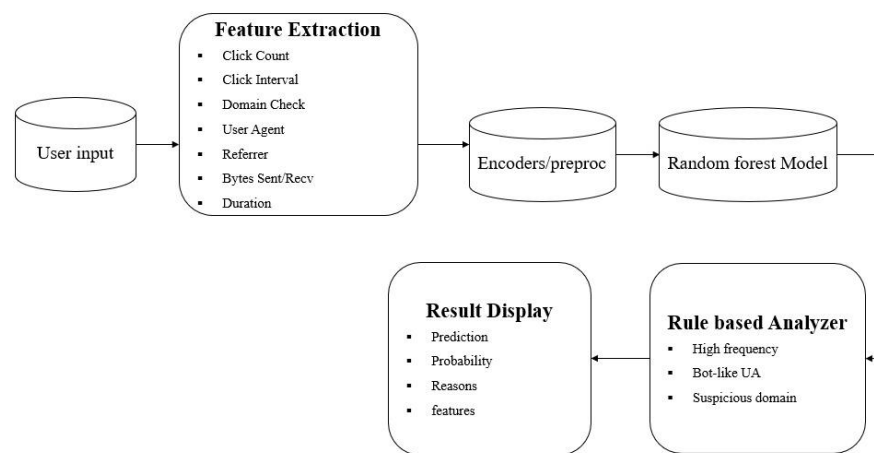


Fig 4.4 Data flow Diagram Level 3

4.3 ALGORITHM FLOWCHART

Input Collection and Preparation

The process begins with the collection of user input data, such as URLs, IDs, referrer information, and user agents, from various user interactions with a system or interface. Gathering this raw data is critical as it serves as the foundation for further analysis and ensures all necessary information is available for processing.

Feature Extraction and Pre-processing

Next, the collected inputs undergo feature extraction, where key attributes relevant to the analysis, like user behavior signals, are identified and isolated. This step is followed by encoding and pre-processing, transforming the raw

features into a suitable format for the machine learning model, ensuring consistency and quality of the input data.

Machine Learning Prediction and Rule Analysis

The processed data is then fed into a Random Forest (RF) model for prediction, enabling the system to determine likely outcomes or classify inputs based on learned patterns. Following this, a rule-based analysis is conducted to interpret or validate the model's results further, applying additional logical checks for more robust, explainable decisions.

Displaying Results

Finally, the outcomes from both the machine learning model and rule-based analyzer are presented to the user, providing clear insights or classifications based on the analysis performed. This completes the workflow, ensuring end-users receive actionable and transparent results.

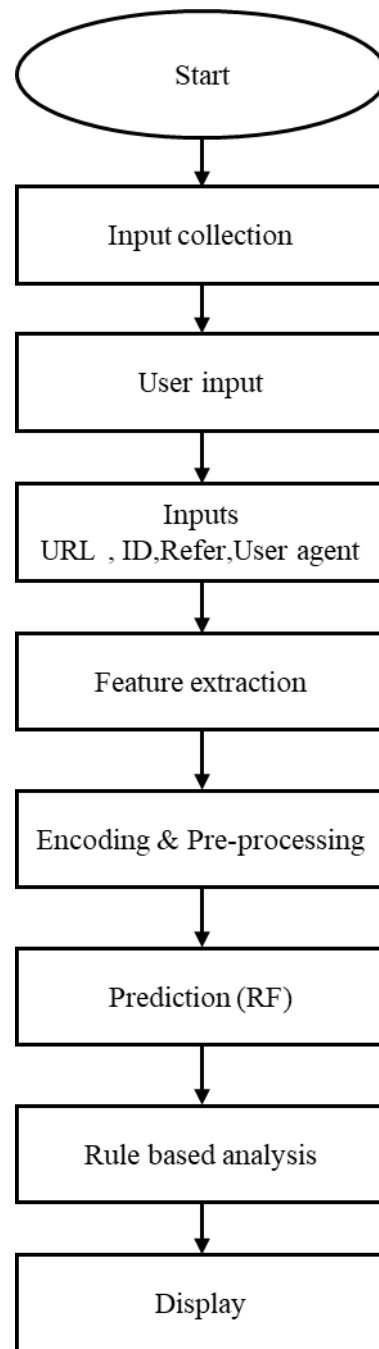


Fig 4.5 Algorithm Flowchart Diagram

The click fraud detection process begins with the Start phase, where the system is initialized and prepared for data collection. The next step is Input Collection, during which data related to user clicks is gathered from various sources such as advertising servers, click logs, and tracking systems. In the User Input stage, specific parameters such as date ranges, campaign IDs, or

datasets are provided by the user to define the scope of analysis. The system then processes key Inputs, which include attributes like the URL, user ID, referrer, and user agent — each of which helps in identifying the origin and nature of each click.

Once the raw data is collected, Feature Extraction is performed to derive meaningful patterns such as the frequency of clicks, IP repetition, time intervals between clicks, and user behavior characteristics. These extracted features are then passed through the Encoding and Pre-processing phase, where categorical data is encoded, missing values are handled, and the dataset is standardized to ensure compatibility with the machine learning model.

The prepared data is then analyzed using the Random Forest (RF) Prediction model, which classifies each click as either legitimate or fraudulent based on patterns learned from historical data. After this, a Rule-Based Analysis step is applied to enhance accuracy — incorporating predefined rules such as detecting multiple clicks from the same IP address or identifying unrealistic location changes. Finally, the results are Displayed to the user through an interface or dashboard, showing detailed insights into detected fraudulent activities and legitimate clicks. This end-to-end process ensures accurate, efficient, and reliable detection of click fraud in digital advertising systems.

CHAPTER V

SYSTEM IMPLEMENTATION

5.1 SOFTWARE DESCRIPTION

5.1.1 HTML

HyperText Markup Language (HTML) is the standard language used for creating and designing web pages. It defines the structure of web content by using a system of tags and elements that tell the browser how to display text, images, links, and other media. HTML acts as the backbone of all websites and works together with CSS (Cascading Style Sheets) and JavaScript to create complete, interactive web experiences. It is a platform-independent, easy-to-learn markup language that forms the foundation for all web development. HTML documents are simply text files that can be created using any text editor and viewed using any web browser.

HTML uses tags enclosed in angle brackets (e.g., `<html>`, `<head>`, `<body>`, `<p>`, `<h1>`). These tags define various elements such as headings, paragraphs, links, images, and tables. The structure of an HTML document follows a hierarchical model, starting from the `<html>` root element and including the `<head>` and `<body>` sections. The `<head>` contains metadata and references to external files like CSS or scripts, while the `<body>` holds all the visible content of the webpage.

HTML has evolved through multiple versions to support modern web requirements. The current version, HTML5, introduces enhanced capabilities such as multimedia support, semantic elements, and improved APIs for application development. It enables developers to create dynamic, responsive, and accessible web pages without relying heavily on external plugins like Flash. HTML5 is also optimized for mobile devices, ensuring a seamless experience across different screen sizes.

Features of HTML

- HTML is a straightforward language that can be easily understood and implemented, even by beginners. Its syntax is clean, readable, and does not require programming logic.
- HTML files can run on any operating system or browser without modification, making it a universally compatible markup language.
- HTML allows linking of documents using the `<a>` tag, enabling easy navigation between web pages and websites.
- HTML5 supports embedding audio, video, and graphical content directly into web pages without requiring external plugins.
- HTML5 introduces semantic tags like `<header>`, `<footer>`, `<article>`, and `<section>` to make the document more meaningful and accessible to search engines and assistive technologies.
- HTML provides forms using `<form>`, `<input>`, `<select>`, and `<textarea>` tags to collect user data efficiently.
- HTML easily integrates with CSS for styling and JavaScript for interactivity, forming the core trio of modern web development.
- HTML5 supports local storage, caching, and offline web applications, enhancing performance and user experience.

5.1.2 CSS

Cascading Style Sheets (CSS) is a stylesheet language used to describe the presentation and layout of HTML documents. While HTML defines the structure and content of a webpage, CSS controls how these elements appear visually, including their colors, fonts, spacing, and positioning. CSS allows developers to separate content from design, making web pages easier to maintain, more consistent, and aesthetically appealing.

CSS works by associating rules with HTML elements. Each rule consists of a selector (which defines which HTML elements the rule applies to) and a declaration block (which specifies style properties and their values). For example, a rule can change the text color of all headings to blue or add padding around paragraphs. CSS supports multiple types of styling: inline (directly in

HTML), internal (within `<style>` tags), and external (via linked .css files). External stylesheets are preferred for larger websites because they promote reusability and consistency across multiple pages.

Over time, CSS has evolved into CSS3, which introduces advanced features such as animations, transitions, flexible box layouts (Flexbox), grid layouts, and media queries for responsive design. These capabilities allow web developers to create highly interactive, visually appealing, and adaptive websites that work seamlessly across different devices and screen sizes. CSS also improves accessibility by providing options to adjust visual cues for users with disabilities.

5.1.3 JAVASCRIPT

JavaScript is a high-level, interpreted programming language primarily used to create interactive and dynamic content on web pages. Unlike HTML, which defines the structure of a webpage, and CSS, which controls its presentation, JavaScript adds behavior and logic to websites, allowing them to respond to user actions, validate forms, manipulate page elements, and communicate with servers in real time. It is an essential part of modern web development and works seamlessly with HTML and CSS.

JavaScript is an event-driven, functional, and object-oriented language. It runs directly in the web browser without the need for compilation, enabling instant execution of code as users interact with a webpage. With the advent of ECMAScript standards, JavaScript has evolved into a robust language capable of supporting complex applications, including single-page applications (SPAs), server-side development with Node.js, and mobile or desktop app development.

Modern JavaScript also supports Document Object Model (DOM) manipulation, which allows developers to dynamically update content, styles, and structure of web pages without reloading them. It can handle asynchronous operations through technologies like AJAX, Promises, and `async/await`, enabling real-time data fetching and smoother user experiences. Additionally,

JavaScript has a rich ecosystem of libraries and frameworks, such as React, Angular, and Vue.js, which simplify development and improve performance.

5.2 ALGORITHM USED: RANDOM FOREST

The proposed Click Fraud Detection System employs the Random Forest algorithm as its primary machine learning model for classification. Random Forest is a powerful ensemble learning technique that operates by constructing multiple decision trees during the training process and combining their outputs to produce a more accurate and stable prediction. Each decision tree independently classifies data samples, and the final output is determined through a majority voting mechanism. This ensemble structure reduces the risk of errors that may arise from individual weak learners, resulting in a robust and generalized model capable of handling complex datasets. In the context of click fraud detection, the Random Forest algorithm analyzes numerous features derived from clickstream data, such as click frequency, IP address patterns, device identifiers, referral sources, and session durations. By examining the relationships between these features, the algorithm learns to differentiate between normal and suspicious click behaviors. For instance, repetitive clicks from a single IP within short intervals or from unusual locations can signal fraudulent intent. Random Forest captures these patterns effectively because it considers multiple combinations of features across different trees, allowing it to detect even subtle anomalies that might go unnoticed by traditional rule-based systems. One of the main advantages of using Random Forest is its ability to handle large datasets with high dimensionality. In online advertising, click data often contains hundreds of attributes, including both numerical and categorical variables. The algorithm handles this diversity efficiently by randomly selecting subsets of features for each tree, ensuring that no single feature dominates the prediction process. This improves model generalization and reduces overfitting, making it suitable for real-time fraud detection where incoming data can vary significantly in structure and distribution. Another important aspect of Random Forest is its feature importance analysis, which

helps identify which attributes contribute the most to the model's decision-making process. In the fraud detection system, this capability provides valuable interpretability by showing users which factors—such as IP repetition, short click intervals, or unusual referrer domains—have the highest influence on the classification outcome. This transparency not only builds trust in the model's predictions but also assists analysts in understanding the behavioral patterns that indicate fraudulent activity. Furthermore, Random Forest performs exceptionally well in terms of accuracy and computational efficiency. During training, it uses techniques like bootstrap sampling and aggregation (bagging) to create diverse models and reduce variance. When deployed, it can process new click records quickly and classify them as either Fraudulent or Legitimate in real time. Combined with its scalability and resilience to noise, Random Forest proves to be an ideal choice for building an AI-powered click fraud detection system. driven decisions while maintaining campaign integrity and reducing financial risks.

Working of Random Forest Algorithm in Click Fraud Detection

Step 1: Dataset Preparation

The process begins with collecting click data from ad servers or tracking systems. The dataset typically includes information such as user ID, IP address, URL, referrer, user-agent, click timestamp, and click frequency. These attributes are used as input features for the model. The data is pre-processed to remove duplicates, handle missing values, and encode categorical data into numerical form.

Step 2: Creating Subsets using Bootstrapping

Random Forest uses a technique called bootstrapping to create multiple subsets of the dataset. Each subset is formed by randomly selecting samples from the original data with replacement. This means some records may appear more than once in a subset, while others may not appear at all. Each subset is then used to train one decision tree in the forest.

Step 3: Building Decision Trees

For each subset, a Decision Tree is constructed. However, instead of considering all features at every split, Random Forest selects a random subset of features. This randomness ensures that each tree learns different patterns from the data. For example, one tree might focus more on IP address patterns, while another might analyze time intervals or user-agent information.

Step 4: Tree Training and Splitting

Each decision tree splits the data based on conditions that best separate legitimate and fraudulent clicks. The splitting continues until a stopping criterion is reached (such as maximum depth or minimum number of samples in a node). Each leaf node of the tree represents a prediction outcome — either “fraud” or “legitimate.”

Step 5: Combining Predictions

Once all trees are trained, they are combined to form the Random Forest model. During prediction, a new input (a click record) is passed through every tree. Each tree independently predicts whether the click is fraudulent or not. The final result is obtained by majority voting — the class (fraud or legitimate) predicted by most trees becomes the final output.

Step 6: Result Interpretation

Along with the prediction, the model can provide feature importance scores showing which features contributed most to the decision. For instance, repeated clicks from the same IP or very short time intervals between clicks may have a higher importance score, indicating possible fraud.

Through these steps, the Random Forest algorithm provides a reliable and accurate way to detect click fraud by combining the strength of multiple decision trees, reducing overfitting, and improving generalization on unseen data.

CHAPTER VI

SYSTEM TESTING

System testing is a critical aspect of Software Quality Assurance and represents the ultimate review of specification, design and coding. Testing is a process of executing a program with the intent of finding an error. A good test is one that has a probability of finding an as yet undiscovered error. The purpose of testing is to identify and correct bugs in the developed system. Nothing is complete without testing. Testing is the vital to the success of the system.

In the code testing the logic of the developed system is tested. For this every module of the program is executed to find an error. To perform specification test, the examination of the specifications stating what the program should do and how it should perform under various conditions. Unit testing focuses first on the modules in the proposed system to locate errors. This enables to detect errors in the coding and logic that are contained within that module alone. Those resulting from the interaction between modules are initially avoided. In unit testing step each module has to be checked separately.

System testing does not test the software as a whole, but rather than integration of each module in the system. The primary concern is the compatibility of individual modules. One has to find areas where modules have been designed with different specifications of data lengths, type and data element name. Testing and validation are the most important steps after the implementation of the developed system. The system testing is performed to ensure that there are no errors in the implemented system. The software must be executed several times in order to find out the errors in the different modules of the system.

Validation refers to the process of using the new software for the developed system in a live environment i.e., new software inside the organization, in order to find out the errors. The validation phase reveals the failures and the bugs in the developed system. It will be come to know about

the practical difficulties the system faces when operated in the true environment. By testing the code of the implemented software, the logic of the program can be examined. A specification test is conducted to check whether the specifications stating the program are performing under various conditions. Apart from these tests, there are some special tests conducted which are given below:

Peak Load Tests: This determines whether the new system will handle the volume of activities when the system is at the peak of its processing demand. The test has revealed that the new software for the agency is capable of handling the demands at the peak time.

Storage Testing: This determines the capacity of the new system to store transaction data on a disk or on other files. The proposed software has the required storage space available, because of the use of a number of hard disks.

Performance Time Testing: This test determines the length of the time used by the system to process transaction data.

In this phase the software developed Testing is exercising the software to uncover errors and ensure the system meets defined requirements. Testing may be done at 4 levels

- Unit Level
- Module Level
- Integration & System
- Regression

6.1 UNIT TESTING

A Unit corresponds to a screen /form in the package. Unit testing focuses on verification of the corresponding class or Screen. This testing includes testing of control paths, interfaces, local data structures, logical decisions, boundary conditions, and error handling. Unit testing may use Test Drivers, which are control programs to co-ordinate test case inputs and outputs, and Test stubs, which replace low-level modules. A stub is a dummy subprogram.

6.2 VALIDATION TESTING

In this requirement established as part of software requirements analysis are validated against the software that has been constructed. Validation testing provides final Assurance that software meets all functional, behavioral and performance requirements. Validation can be defined in many ways but a simple definition is that validation succeeds when software Function in a manner that can be reasonably by the customer.

Validation test criteria

Configuration review

Alpha and Beta testing (conducted by end user)

6.3 MODULE LEVEL TESTING

Module Testing is done using the test cases prepared earlier. Module is defined during the time of design.

6.4 INTEGRATION & SYSTEM TESTING

Integration testing is used to verify the combining of the software modules. Integration testing addresses the issues associated with the dual problems of verification and program construction. System testing is used to verify, whether the developed system meets the requirements. System testing is actually a series different test whose primary purpose is to full exercise the computer base system. Where the software and other system elements are tested as whole. To test computer software, we spiral out along streamlines that broadens the scope of testing with each turn.

The last higher-order testing step falls outside the boundary of software Engineering and in to the broader context of computer system engineering. Software, once validated, must be combining with order system Elements (e.g. hardware, people, databases). System testing verifies that all the elements Mesh properly and that overall system function/performance is achieved.

1.Recovery Testing

2.Security Testing

3.Stress Testing

6.5 REGRESSION TESTING

Each modification in software impacts unmodified areas, which results in serious injuries to that software. So, the process of re-testing for rectification of errors due to modification is known as regression testing.

Installation and Delivery: Installation and Delivery is the process of delivering the developed and tested software to the customer. Refer the support procedures.

Acceptance and Project Closure: Acceptance is the part of the project by which the customer accepts the product. This will be done as per the Project Closure, once the customer accepts the product, closure of the project is started. This includes metrics collection, PCD, etc.

CHAPTER VII

RESULT AND DISCUSSION

The system analyzes various input features to evaluate and detect suspicious advertisement links effectively. It primarily uses the URL as the main input, examining characteristics such as URL length, number of digits, and special characters to identify patterns commonly associated with phishing or fraudulent activities. Longer URLs and excessive digits or special symbols often indicate attempts to disguise malicious parameters or redirect users to unsafe sites. By collectively analyzing these structural and behavioral features, the model performs a comprehensive assessment to distinguish between genuine and suspicious links. This feature-driven approach enhances detection accuracy, enabling automated decision-making for digital advertising platforms. As a result, the system not only safeguards users from engaging with unsafe or deceptive advertisements but also helps maintain the credibility of legitimate advertisers by filtering out low-quality or harmful links before they reach a broader audience.

Confusion Matrix

The Confusion Matrix is a vital performance evaluation tool used in the Cyber Attack Prediction System, helping to measure how accurately the model distinguishes between normal network activities and cyber-attack incidents. It provides a detailed comparison between the actual attack labels (from the dataset) and the predicted labels (generated by the machine learning or generative AI model). This evaluation enables researchers and developers to understand not just the accuracy, but also the types of errors made by the model.

Predicted / Actual	Actual Attack	Actual Normal
Predicted Attack	True Positive (TP) – correctly detected cyber-attacks	False Positive (FP) – normal activities wrongly classified as attacks
Predicted Normal	False Negative (FN) – actual attacks misclassified as normal	True Negative (TN) – correctly identified normal network activities

Table 7.1 confusion matrix

In this context,

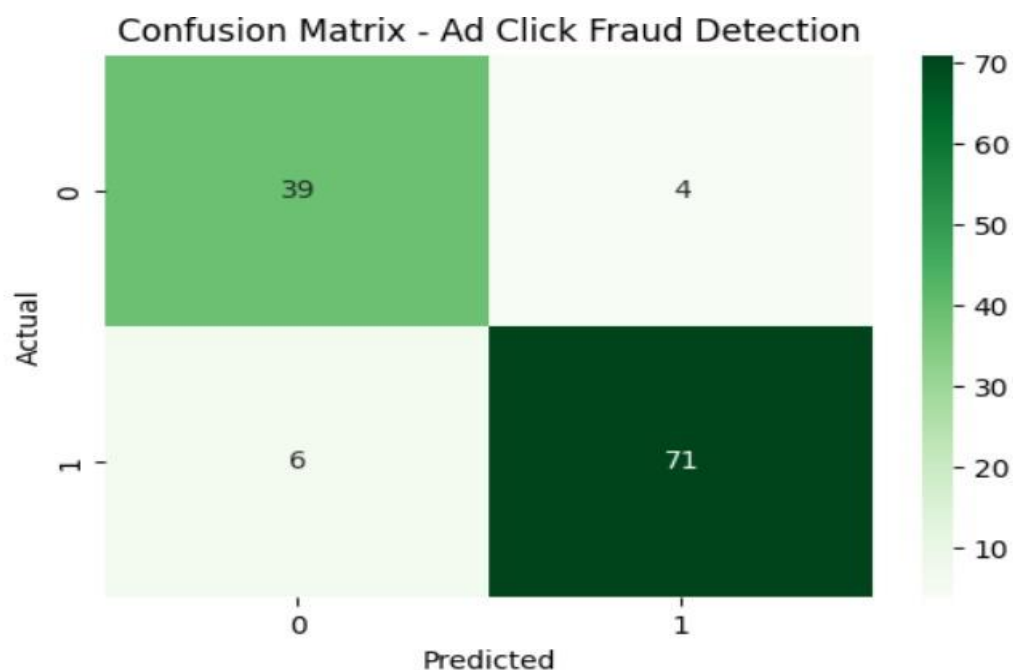
- **True Positive (TP)** represents the number of real cyber-attacks correctly identified by the system.
- **True Negative (TN)** represents legitimate or safe network traffic correctly recognized as normal.
- **False Positive (FP)** occurs when normal traffic is mistakenly flagged as an attack, leading to unnecessary alerts.
- **False Negative (FN)** represents missed detections — where an actual attack goes undetected, posing a major security risk.

The effectiveness of the model is evaluated using key metrics derived from the confusion matrix, such as:

- **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$
- **Precision** = $TP / (TP + FP)$
- **Recall (Detection Rate)** = $TP / (TP + FN)$
- **F1-Score** = $2 \times (Precision \times Recall) / (Precision + Recall)$

These parameters collectively reflect how efficient the model is in detecting attacks while minimizing false alarms. In traditional machine learning models,

such as Random Forest, SVM, or Logistic Regression, the confusion matrix helps evaluate static, rule-based detection accuracy. However, when shifting toward Generative Artificial Intelligence (GAI) approaches, the confusion matrix becomes even more significant — as GAI models learn dynamic attack behaviors and generate synthetic samples to improve threat detection accuracy. Thus, the confusion matrix serves as a core evaluation component bridging traditional ML and next-generation AI-driven cybersecurity frameworks. It ensures that the system not only achieves high accuracy but also maintains a balance between sensitivity (attack detection) and specificity (normal activity recognition), making it a robust and reliable cyber defense solution



Confusion matrix

Classification Report:				
	precision	recall	f1-score	support
Legitimate	0.87	0.91	0.89	43
Fraud	0.95	0.92	0.93	77
accuracy			0.92	120
macro avg	0.91	0.91	0.91	120
weighted avg	0.92	0.92	0.92	120

Classification report

✅ Model training complete!

Accuracy : 91.67%

Precision : 94.67%

Recall : 92.21%

F1 Score : 93.42%

Performance metrics

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 CONCLUSION

The implementation of a cybersecurity framework for real-time ad click fraud detection is essential in today's rapidly evolving digital advertising ecosystem. The proposed framework demonstrates that integrating advanced detection mechanisms, including machine learning algorithms, anomaly detection models, and real-time monitoring systems, significantly enhances the ability to identify fraudulent activities and mitigate financial losses. By analyzing click patterns, user behavior, and network activities, the framework can differentiate between genuine user interactions and fraudulent clicks, ensuring advertisers receive accurate campaign performance metrics.

Furthermore, the adoption of a layered cybersecurity approach strengthens overall system resilience, reducing vulnerabilities that fraudsters often exploit. Real-time detection not only prevents immediate financial damage but also protects the integrity of advertising platforms, maintains trust among stakeholders, and optimizes resource allocation for marketing campaigns. The framework's flexibility allows it to adapt to evolving fraud techniques, ensuring long-term sustainability and scalability.

In conclusion, the study validates that combining artificial intelligence, continuous monitoring, and robust cybersecurity policies forms an effective defense against click fraud. Future work can focus on enhancing predictive analytics, incorporating blockchain for transparent ad transactions, and expanding cross-platform detection capabilities. This proactive approach ultimately promotes a secure, efficient, and trustworthy digital advertising environment.

8.2 FUTURE ENHANCEMENT

Future work can focus on enhancing predictive analytics to improve early fraud detection, incorporating blockchain technology for transparent and tamper-proof ad transactions, and expanding cross-platform detection capabilities to provide comprehensive protection across multiple advertising channels. This proactive approach will further promote a secure, efficient, and trustworthy digital advertising environment.

APPENDIX A

SOURCE CODE

```
from django.shortcuts import render

import pandas as pd

import pickle

import os

from django.conf import settings


def home(request):

    return render(request, "index.html")

# detector/views.py

from django.shortcuts import render

from django.conf import settings

import os, pickle

from urllib.parse import urlparse, parse_qs

import pandas as pd

import numpy as np


MODEL_PATH = os.path.join(settings.BASE_DIR,
"realtime_fraud_model.pkl")


def extract_url_features(url):

    parsed = urlparse(url)
```

```

domain = parsed.netloc

path_len = len(parsed.path)

qs = parse_qs(parsed.query)

num_params = len(qs)

has_utm = int(any(k.startswith("utm") for k in qs.keys()))

return {"domain": domain, "path_len": path_len, "num_params":
num_params, "has_utm": has_utm}

def load_bundle():

    with open(MODEL_PATH, "rb") as f:

        data = pickle.load(f)

    if not isinstance(data, dict):

        raise ValueError("Loaded model file is not a dictionary bundle.")

    return data

def paste_link(request):

    return render(request, "paste_link.html")

import numpy as np

import pickle

from django.shortcuts import render

import numpy as np

import pandas as pd

```

```
import pickle

from django.shortcuts import render

def predict_link(request):
    context = {}

    if request.method == "POST":
        try:
            # Load model bundle

            with open("realtime_fraud_model.pkl", "rb") as f:

                bundle = pickle.load(f)

            if not isinstance(bundle, dict):

                raise ValueError("Loaded model file is not a dictionary bundle.")

            model = bundle['model']

            encoders = bundle.get('encoders', {})

            feature_columns = bundle.get('features', [])

            # Get inputs from form

            url = request.POST.get("url", "").strip()

            ref = request.POST.get("referrer", "").strip() or "none"

            ua = request.POST.get("user_agent", "").strip() or "unknown"

            click_count = int(request.POST.get("click_count", 1))
```

```
click_interval = float(request.POST.get("click_interval", 0.5))  
suspicious_domain = int(request.POST.get("suspicious_domain", 0))
```

```
# Numeric features (fill defaults if missing)
```

```
bytes_sent = float(request.POST.get("bytes_sent", 0))
```

```
bytes_received = float(request.POST.get("bytes_received", 0))
```

```
duration = float(request.POST.get("duration", 0))
```

```
# Build row dict
```

```
row = {  
    "click_count": click_count,  
    "click_interval": click_interval,  
    "suspicious_domain": suspicious_domain,  
    "ua": ua,  
    "ref": ref,  
    "bytes_sent": bytes_sent,  
    "bytes_received": bytes_received,  
    "duration": duration  
}
```

```
# Encode categorical columns safely
```

```
for col in feature_columns:
```

```
    if col not in row:
```

```

        # Missing column, add default 0

        row[col] = 0

    elif col in encoders:

        le = encoders[col]

        try:

            row[col] = int(le.transform([row[col]])[0])

        except Exception:

            row[col] = -1 # unseen category

# Convert row to DataFrame with correct column order

X_pred = pd.DataFrame([row], columns=feature_columns)

X_pred = X_pred.fillna(0)

# Predict

pred_label = model.predict(X_pred)[0]

proba = model.predict_proba(X_pred)[0][1] if hasattr(model,
"predict_proba") else None

# Build rule-based reasons

reasons = []

if row.get("suspicious_domain", 0) == 1:

    pred_label = model.predict(X_pred)[0]

```

APPENDIX B

SCREENSHOTS

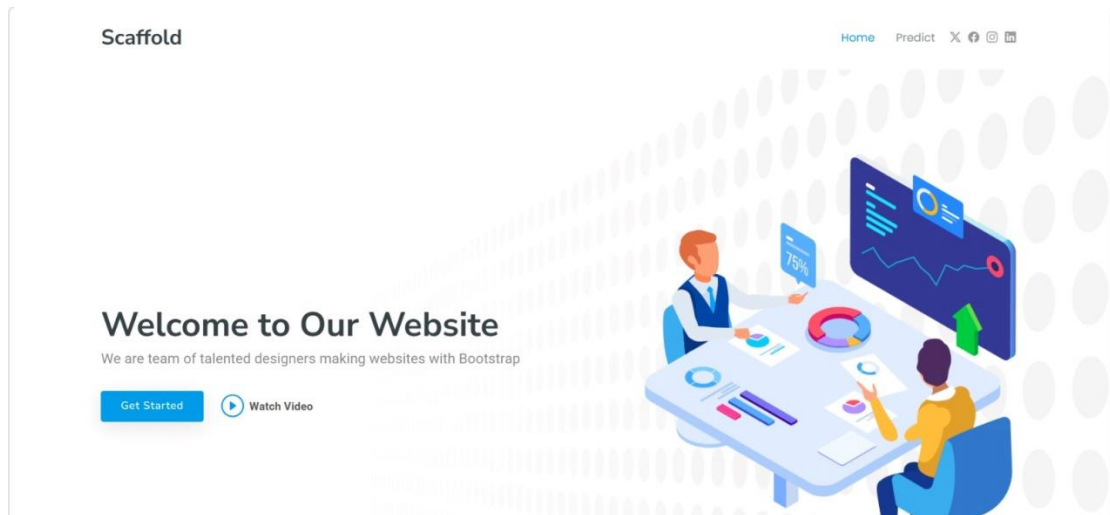


Fig 1: Home Page

Paste ad URL to check (real-time)

Ad URL

IP Address (optional)

Referrer (optional)

User Agent (optional)

Fig 2: Updated Form

Paste ad URL to check (real-time)

Ad URL

IP Address (optional)

Referrer (optional)

User Agent (optional)

Fig 3: Prediction Page

 **Ad Click Fraud Detection Result**

URL Scanned:
<https://cheapclicks.biz/ad/4752>

Prediction:

Fraudulent

Risk Probability:

84.0%

Reasons:

- Clicks are extremely frequent (short interval or very high click count).
- User agent looks like a bot or crawler.

Extracted Features:

Click Count:	181
Click Interval:	0.88
Suspicious Domain:	0
User Agent:	curl/7.68.0
Referrer:	https://newsletter.example.com
Bytes Sent:	0.0

Fig 4: Fraud Detection

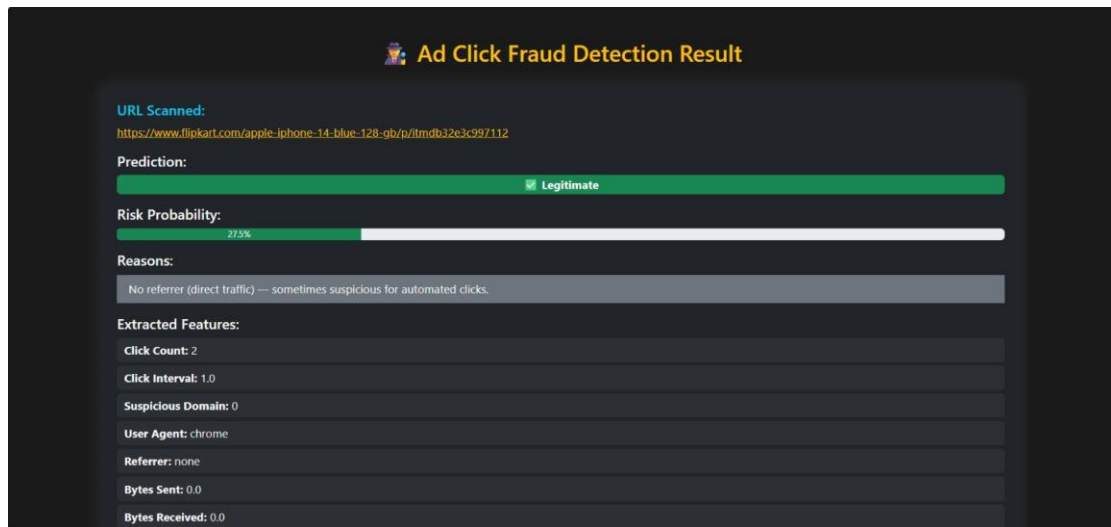


Fig 5: Legitimate detection

REFERENCES

1. A. Batool and Y.-C. Byun, “An ensemble architecture based on deep learning model for click fraud detection in Pay-Per-Click advertisement campaign,” *IEEE Access*, vol. 10, pp. 113410–113426, 2022, doi: 10.1109/ACCESS.2022.3211528.
2. A. Dash and S. Pal, “Auto-detection of click-frauds using machine learning Auto-detection of click-frauds using machine learning,” *Int. J. Eng. Sci. Comput.*, vol. 10, pp. 27227–27235, Sep. 2020.
3. A. K. Wood and A. M. Ravel, “Fool me once: Regulating fake news and other online advertising,” *S. Cal. L. Rev.*, vol. 91, p. 1223, Jan. 2017.
4. A. Purwar, A. K. Jain, I. Chawla, I. Gupta, M. Raj, and D. Jain, “Click fraud detection using ensemble classifier,” in *Proc. Int. Conf. Artif.-Bus. Anal., Quantum Mach. Learn.*, Jan. 2024, pp. 15–23.
5. B. Kirkwood, M. Vanamala, and N. Seliya, “Click fraud detection of online advertising using machine learning algorithms,” in *Proc. IEEE Int. Conf. Electro Inf. Technol. (eIT)*, May 2024, pp. 586–590. [Online]. Available: <https://api.semanticscholar.org/CorpusID>
6. B. Stone-Gross, R. Stevens, A. Zarras, R. Kemmerer, C. Kruegel, and G. Vigna, “Understanding fraudulent activities in online ad exchanges,” in *Proc. ACM SIGCOMM Conf. Internet Meas. Conf.*, Nov. 2011, pp. 279–294.
7. C. Phua, E.-Y. Cheu, G.-E. Yap, K. Sim, and M.-N. Nguyen, “Feature engineering for click fraud detection,” in *Proc. Work. Fraud Detect. Mob. Advert.*, 2012, pp. 1–10. [Online]. Available: <http://palanteer.sis.smu.edu.sg/fdma2012/doc/FirstWinner-Starrystarrynight-Paper.pdf%5Cnpapers2://publication/uuid/9290A6CF-A861-4058-99F4-D39706B0619A>
8. D. Berrar, “Random forests for the detection of click fraud in online mobile advertising,” in *Proc. Int. Work. Fraud Detect. Mob. Advert(FDMA)*,

Singapore, 2012, pp. 1–10. [Online]. Available:http://berrar.com/resources/Berrar_FDMA2012.pdf

9. D. Sisodia, D. S. Sisodia, and D. Singh, “Evaluating feature importance to investigate publishers conduct for detecting click fraud,” in *Machine Intelligence Techniques for Data Analysis and Signal Processing (Lecture Notes in Electrical Engineering)*, vol. 997. Berlin, Germany: Springer, 2023, pp. 515–524, doi: 10.1007/978-981-99-0085-5_42.

10. D. Sisodia and D. S. Sisodia, “Gradient boosting learning for fraudulent publisher detection in online advertising,” *Data Technol. Appl.*, vol. 55, no. 2, pp. 216–232, Apr. 2021, doi: 10.1108/dta-04-2020-0093.

11. D. Sisodia and D. S. Sisodia, “Quad division prototype selection- based Knearest neighbor classifier for click fraud detection from highly skewed user click dataset,” *Eng. Sci. Technol., Int. J.*, vol. 28, Apr. 2022, Art. no. 101011, doi: 10.1016/j.jestch.2021.05.015.

12. D. Sisodia and D. S. Sisodia, “Stacked generalization architecture for predicting publisher behaviour from highly imbalanced user-click data set for click fraud detection,” *New Gener. Comput.*, vol. 41, no. 3, pp. 581–606, Sep. 2023, doi: 10.1007/s00354-023-00218-1.

13. E.-A. Minastireanu and G. Mesnita, “Light GBM machine learning algorithm to online click fraud detection,” *J. Inf. Assurance Cybersecur.*, vol. 2019, pp. 1–12, Apr. 2019, doi: 10.5171/2019.263928

14. J. H. Yan and W. R. Jiang, “Research on information technology with detecting the fraudulent clicks using classification method,” *Adv. Mater. Res.*, vol. 859, pp. 586–590, Dec. 2013, doi: 10.4028/www.scientific.net/amr.859.586.

15. Juniper Research, Hampshire, U.K. Quantifying the Cost of AdFraud: 2023–2028. Accessed: Jul. 12, 2024. [Online]. Available:https://fraudblocker.com/wp-content/uploads/2023/09/Ad-Fraud-Whitepaper_Juniper-Research.pdf

16. K. S. Perera, B. Neupane, M. A. Faisal, Z. Aung, and W. L. Woon, “A novel ensemble learning-based approach for click fraud detection in mobile advertising,” in *Mining Intelligence and Knowledge Exploration (Lecture Notes in Computer Science: Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 8284. Berlin, Germany: Springer, 2013, pp. 370–382, doi: 10.1007/978-3-319-03844-5_38.
17. L. Singh, D. Sisodia, K. Shashvat, A. Kaur, and P. C. Sharma, “A reliable click-fraud detection system for the investigation of fraudulent publishers in online advertising,” in *Applied Intelligence in Human-Computer Interaction*. Boca Raton, FL, USA: CRC Press, Jul. 2023.
18. M. Aljabri and R. M. A. Mohammad, “Click fraud detection for online advertising using machine learning,” *Egyptian Informat. J.*, vol. 24, no. 2, pp. 341–350, Jul. 2023, doi: 10.1016/j.eij.2023.05.006.
19. R. A. Alzahrani and M. Aljabri, “AI-based techniques for ad click fraud detection and prevention: Review and research directions,” *J. Sensor Actuator Netw.*, vol. 12, no. 1, p. 4, Dec. 2022, doi:10.3390/jsan12010004.
20. R. Dekou, S. Savo, S. Kufeld, D. Francesca, and R. Kawase, “Machine learning methods for detecting fraud in online marketplaces,” in *Proc. CEUR Workshop*, vol. 3052, Jan. 2021, pp. 3–7.
21. R. Mouawi, M. Awad, A. Chehab, I. H. E. Hajj, and A. Kayssi, “Towards a machine learning approach for detecting click fraud in mobile advertizing,” in *Proc. Int. Conf. Innov. Inf. Technol. (IIT)*, Nov. 2018, pp. 88–92, doi: 10.1109/INNOVATIONS.2018.8605973.
22. S. Shaik and V. Kakulapati, “Fraud detection of AD clicks using machine learning techniques,” *J. Sci. Res. Rep.*, vol. 29, no. 7, pp. 84–89, Jun. 2023, doi: 10.9734/jsrr/2023/v29i71762.

23. (2024). Wasted Ad Spend Report 2024. [Online] Available: https://lplunio.ai/wpcontent/uploads/2023/09/Lunio_Wasted_Ad_SpendReport_2024_V2.pdf
24. X. Zhu, H. Tao, Z. Wu, J. Cao, K. Kalish, and J. Kayne, Fraud Prevention in Online Digital Advertising. Cham, Switzerland: Springer, 2017.